

人智导论复习

人智导论复习 202501 2211030

现在我面临着很尴尬的处境，我只有不到20h来复习这门课程了，或者应该叫预习。

简介

什么是人工智能？定义？

示例：研究如何用计算机来表示和执行人类的智能活动

自动控制、科学计算等固定算法的都不算人工智能

人工智能三大流派

- 符号主义
 - 基于逻辑和符号推理，认为智能可以通过**符号操作**实现，把知识表达为离散的符号和规则，进行明确的逻辑推理。
 - 生产系统、专家系统、图灵测试、逻辑推理
- 连接主义
 - 模仿人脑神经网络的结构与功能，通过**神经网络的连接和权重调整**来实现智能。
 - 人工神经网络（ANNs）、深度学习（如CNN、RNN）
- 行为主义
 - 强调通过与环境的交互来表现智能行为，关注**行为适应性**而非内在表示，不追求对智能的内部机制建模，而是通过**学习行为策略**解决问题。
 - 强化学习（RL）、进化算法（EA）、游戏AI（如AlphaGo）

人工智能系统的知识表示中，包含以下主要要素：

1. 事实：
 - 表示客观存在的信息或具体陈述，例如“猫是哺乳动物”。
2. 规则：
 - 用于描述知识的推理过程或条件反应关系，例如“如果 X 是猫，则 X 是动物”。
3. 控制和元知识：
 - 控制知识是指导系统如何高效推理和使用知识的规则。
 - 元知识是关于知识本身的知识，例如知识的结构、来源或适用范围。

1. 状态空间法

- 主要通过构建状态空间（如图或树），逐步探索和搜索解答路径，适合解决搜索类问题（如路径规划、游戏等）。

2. 谓词逻辑法

- 使用谓词逻辑进行问题描述和推理，多用于知识表示与推理。

3. 语义网络法

- 基于语义网络（节点和关系的图）来表示知识，常用于人工智能中的语义表示，。

4. 问题规约法

- 将复杂问题通过分解和转化，最终归约为一组更简单的子问题的方法。这些子问题的解可以直接获得或更容易解决，从而通过子问题的解组合来解决原始问题。

专家系统的组成部分：人机交互界面、知识库、推理机、解释器、综合数据库、知识获取。必须通过人为更新知识库，依赖知识库

产生式系统的推理主要包括以下三种类型：

1. 正向推理 (Forward Chaining)：

- 从已知的事实出发，依据规则不断推导出新的事实，直到得出目标结论。
- 常用于数据驱动的问题求解。

2. 逆向推理 (Backward Chaining)：

- 从目标出发，依据规则逆向追溯所需的条件，验证是否可以由已知事实推导出来。
- 常用于目标驱动的推理过程。

3. 双向推理 (Bidirectional Chaining)：

- 结合正向推理和逆向推理，从已知事实和目标两端同时推理，直至在某个中间状态会合。
- 常用于复杂问题求解。

1. **图灵** 被称为人工智能之父，曾提出一个机器智能的测试模型（图灵测试）。

2. 从已知事实出发，通过规则库求得结论的产生式系统的推理方式是 **正向推理**。

3. 在诸如走迷宫、下棋、八数码等游戏中，常用到的一种人工智能的核心技术称为 **搜索** 技术，解这类问题时，常把在迷宫的位置、棋的布局、八数码所排成的形势用图来表示，这种图称为 **状态空间图**。

4. 在 **启发式** 搜索当中，通常用启发函数来表示启发性信息。

5. **宽度优先搜索** 是一种盲目搜索方法，该方法每次选择最浅的节点优先进行搜索。

6. 表示在规则中，证据A为真下结论B为真的 **规则**。

7. **占优策略** 是指无论其他参与者采取什么策略，某参与者采用该策略的结果都优于或不劣于其他策略。

8. **非零和博弈** 中智能体交互动作的总收益和损失可以小于或大于零。

9. 在 Alpha-Beta 剪枝算法中，我们把一个节点可能取值的上界记作 **Beta** 值。

图灵实验

由图灵提出的一种测试人工智能是否具备人类智能的方法如果一个计算机能够在对话中与人类进行交流，并且人类无法区分对方是计算机还是另一个人，那么这个计算机就可以被认为具备智能。

图灵机

图灵机是一种理论计算模型，由艾伦·图灵于1936年提出，用于形式化描述计算过程。它由一个无限长的纸带、一个读写头、有限的状态集合和一组转移规则组成。图灵机通过读取纸带上的符号，根据当前状态和符号决定执行操作（如写入符号、移动纸带、改变状态）。图灵机的核心思想是它能够模拟任何可计算的过程，因此它是**图灵完备**的，成为现代计算理论的基础。

图灵完备

指一种计算系统或编程语言的能力，即它能够模拟任何图灵机所能执行的计算任务。换句话说，如果一个计算模型或语言能够模拟图灵机的行为，那么它就被称为图灵完备。

搜索

深度优先搜索 (DFS)

深度优先搜索是一种尽可能深入每个分支的搜索策略。它从图的起始节点开始，沿着一条路径不断向下探索，直到没有更多未访问的邻接节点为止，然后回溯到前一个节点，继续探索其他分支，直到图中的所有节点都被访问。

==适合寻找最短路径，尤其在无权图中，能够逐层遍历节点，但空间消耗较大。==

工作原理

- 从起始节点开始，首先访问一个节点。
- 然后，选择一个邻接节点进行递归探索，继续沿着该路径向下深入。
- 如果当前节点没有未访问的邻接节点，则回溯到上一节点，继续搜索其它未访问的邻接节点。
- 当所有节点都被访问过时，搜索结束。

特点

- **数据结构**：通常使用栈（可以是递归调用栈或显式的栈）来管理未访问的节点。
- **搜索顺序**：深度优先搜索会先探查一个分支的所有节点，然后才会回到父节点进行其他分支的搜索。
- **适用场景**：
 - 适用于需要查找每一条路径或寻找图中的所有可能路径的问题。
 - 适合用于求解连通性、拓扑排序等问题。
 - 适用于空间资源有限的情况，尤其是在路径较长时，因为它较少需要存储未探索节点。

时间复杂度

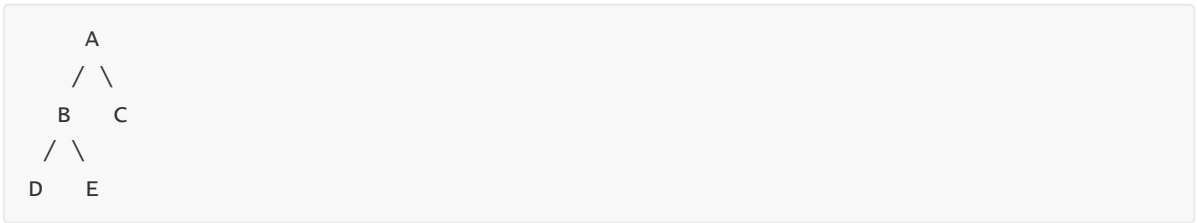
- 对于图中的每个节点和边都访问一次，时间复杂度为 $O(V+E)$ ，其中 V 是节点数， E 是边数。

缺点

- 深度优先搜索可能会陷入无限循环，尤其是在图中存在环的情况下（如果没有标记已访问节点，则容易重复访问）。
- 在某些情况下，深度优先搜索可能会找到非最优解，特别是在寻找最短路径的情况下。

示例

假设图如下：



DFS的遍历顺序可能是： $A \rightarrow B \rightarrow D \rightarrow E \rightarrow C$ 。

宽度优先搜索（BFS）

宽度优先搜索是一种逐层遍历节点的策略。它从图的起始节点开始，首先访问所有与起始节点直接相连的节点，然后再访问这些节点的邻接节点，依此类推，逐层扩展。

==适合探索每一条路径，尤其在空间有限的情况下，能够快速找到解，但可能不总是最优解。==

工作原理

- 从起始节点开始，将其加入到队列中。
- 从队列中取出一个节点，访问它的所有邻接节点。
- 对于每一个邻接节点，如果尚未访问，则将它加入队列。
- 继续从队列中取出下一个节点，直到队列为空。

特点

- **数据结构：**通常使用队列（FIFO）来管理待访问的节点。
- **搜索顺序：**宽度优先搜索会首先遍历起始节点的所有邻接节点，然后再依次遍历它们的邻接节点，即按层次逐层访问。
- **适用场景：**
 - BFS能够找到最短路径，特别是在无权图中寻找从起点到目标的最短路径时非常有效。
 - 适用于寻找图中的最短路径、最小生成树等问题。

时间复杂度

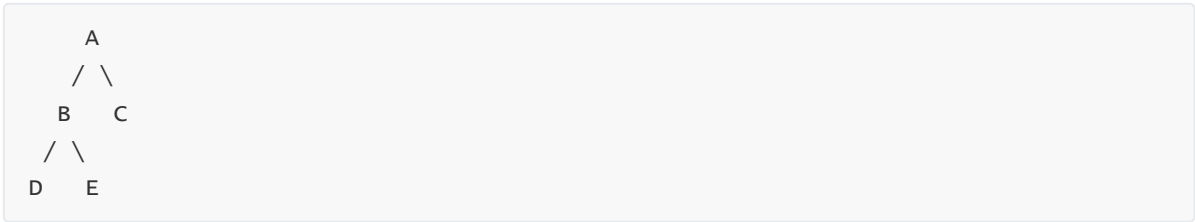
- 对于图中的每个节点和边都访问一次，时间复杂度为 $O(V+E)$ ，其中 V 是节点数， E 是边数。

缺点

- BFS需要存储所有当前层次的节点，可能会消耗较多内存，尤其是在图很大的时候。
- BFS在某些情况下可能不适用于图的深度过大的场景，因为需要维护一个较大的队列。

示例

假设图如下：



BFS的遍历顺序可能是： $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$ 。

A*搜索

A* (A-star) 搜索算法是一种广泛使用的启发式搜索算法，特别适用于路径规划问题，如在图形中找到从起点到目标点的最短路径。它结合了**最佳优先搜索**和**Dijkstra算法**的优点，能够高效地找到最优路径。

A*算法通过使用启发式函数 (heuristic function) 来优化搜索过程。它在每一步都选择一个估计总成本最小的节点来扩展，从而加速搜索过程。具体来说，A*算法评估每个节点的总代价 $f(n)$ ，其中：

$$f(n)=g(n)+h(n)$$

- **$g(n)$** ：从起点到节点 n 的实际成本（即路径长度或移动代价）。
- **$h(n)$** ：从节点 n 到目标的启发式估计成本（即预测的剩余路径成本）。

A*搜索通过综合考虑已走过的路径成本 $g(n)$ 和到目标的估计成本 $h(n)$ ，以此来选择最有希望的路径。

算法步骤

1. **初始化**：将起点添加到开放列表 (open list) 中，开放列表用于存储待检查的节点。每个节点都包含其坐标、成本值和父节点信息。
2. **循环迭代**：
 - 从开放列表中选择具有最小 $f(n)$ 值的节点进行扩展。
 - 对于当前节点的每个相邻节点，计算其 $g(n)$ 、 $h(n)$ 和 $f(n)$ 值。
 - 如果相邻节点不在开放列表中，将其加入开放列表；如果它已经在开放列表中，并且新路径更优（即 $g(n)$ 更小），则更新它的值。
 - 当一个节点的扩展失败（即所有邻接节点都被访问或不可行），搜索将回溯到该节点的父节点，继续探索其他未探索的路径。
3. **结束条件**：
 - 如果目标节点被添加到闭合列表 (closed list) 中，或是选择的节点就是目标节点，则搜索结束，返回路径。
 - 如果开放列表为空，说明没有找到路径，搜索失败。

启发式函数 $h(n)$

启发式函数的选择直接影响算法的效率和准确性。常见的启发式函数有：

- **曼哈顿距离 (Manhattan Distance)**：适用于网格上只有上下左右方向移动的情况，计算节点与目标点在水平和垂直方向上的总距离。

$$h(n) = |x_1 - x_2| + |y_1 - y_2| \quad h(n) = |x_1 - x_2| + |y_1 - y_2|$$

- **欧几里得距离 (Euclidean Distance)**：适用于可以任意方向移动的情况，计算节点与目标点的直线距离。

$$h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} \quad h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

- **切比雪夫距离 (Chebyshev Distance)**：适用于可以在八个方向上移动的情况，计算节点与目标点的最大横纵向距离。

$$h(n) = \max(|x_1 - x_2|, |y_1 - y_2|) \quad h(n) = \max(|x_1 - x_2|, |y_1 - y_2|)$$

优缺点

优点：

- 能够找到最短路径（如果启发式函数是可接受的，即不会过于乐观）。
- 具有较高的效率，尤其是在大规模搜索空间中，通过启发式函数减少了不必要的搜索。

缺点：

- 需要大量的内存，因为开放列表和闭合列表都需要存储大量节点。
- 启发式函数的设计对算法的性能和效率有很大影响，选择不当可能导致性能低下。

博弈论

对抗搜索通常称为博弈搜索

博弈的构成要素

- 参与人 (players)
- 行动 (actions)
- 信息 (information)
- 策略 (strategies)（直接决定玩家收益）
- 收益 (payoffs)
- 均衡 (equilibria)

博弈六特征

- 2 或多玩家
- 轮流vs.同时
- 完整信息vs.不完整信息
- 确定的vs.随机的
- 合作vs.竞争
- 零和vs.非零和

零和博弈

一方的收益必然意味着另一方的损失，博弈各方的收益和损失相加的总和永远为0

纳什均衡

若任何参与者单独改变策略都不会得到好处，则该情形下的策略组合就是一个纳什均衡

例：囚徒困境两人都选择交代罪行

占优策略均衡

在一个博弈中，某一方的策略在不考虑其他玩家策略的情况下，总是能够带来最优的结果。也就是说，对于某个玩家而言，不管其他玩家如何选择自己的策略，选择**占优策略**始终是最优选择。如果所有玩家在博弈中都选择了自己的占优策略，且这种组合策略满足所有玩家的最佳选择，那么这个组合就构成了**占优策略均衡**。

纯策略 vs. 混合策略

纯策略：玩家明确选择一种策略，不包含随机性。

- 例如：在石头剪刀布中，总是出“石头”。

混合策略：玩家根据一定概率分布随机选择策略。

- 例如：在石头剪刀布中，分别以1/3的概率选择“石头”、“剪刀”或“布”。

囚徒困境

假设有两个囚徒因涉嫌共同犯罪被逮捕，他们被单独关押，无法交流。检察官为他们提供以下选择：

- **合作（保持沉默）**：两人都不供认，获得较轻的刑期。
- **背叛（招供）**：一人供认并指证对方，可获得减刑，而对方受重罚。
- **双重背叛**：如果两人都供认，双方都受到中等刑罚。

特点：纳什均衡

- 双方选择背叛是囚徒困境的纳什均衡（即在没有外界干预的情况下，双方都倾向于背叛，因为背叛对个体收益更优）。
- 但这种选择并不是最优的集体结果。

Minimax算法

在Minimax搜索中，我们从根节点开始递归地搜索整个博弈树。每次递归调用会展开一个游戏状态，计算该状态下的最优得分，直到达到树的叶节点。

1. **Max玩家**（Max层）：选择使得自己的得分最大化的分支。
2. **Min玩家**（Min层）：选择使得Max玩家的得分最小化的分支。

最终，当递归回溯到根节点时，Max玩家会选择得分最大的路径，从而确保自己获得最优的策略。

α - β 剪枝

[Alpha-Beta 剪枝 - OI Wiki](#)

==**Alpha剪枝**是针对极大节点（Max节点）进行的优化，而**Beta剪枝**是针对极小节点（Min节点）进行的优化==

基本思想

在对抗搜索中：

- α 是当前已经发现的 **Max 层的最佳值**（最大值的下界）。
- β 是当前已经发现的 **Min 层的最佳值**（最小值的上界）。

如果某个分支的值已经确定不会影响最终决策，则可以提前停止对该分支的搜索（剪枝），避免浪费计算资源。

剪枝条件

- 对于 **Max 层**，如果某个子节点的值 $\geq$ 当前 β ，则不需要继续搜索该节点的其他子分支，因为 Min 玩家不会选择导致 Max 玩家得分更高的路径。
- 对于 **Min 层**，如果某个子节点的值 $\leq$ 当前 α ，则不需要继续搜索该节点的其他子分支，因为 Max 玩家不会选择导致自己得分更低的路径。

蒙特卡洛树搜索（MCTS）

MCTS 通常由以下四个步骤组成：

1. 选择（Selection）

- 从根节点开始，根据搜索树中已有的统计信息，递归地选择下一个要探索的节点，直到到达一个需要扩展的节点。
- 使用 **UCT（Upper Confidence Bound for Trees）** 等公式平衡探索和利用。

2. 扩展（Expansion）

- 如果当前节点未被完全扩展，则为当前节点添加一个新的子节点，代表一种新的可能行动。

3. 模拟（Simulation）

- 从新扩展的节点开始，通过随机模拟完整的游戏（或执行若干步），评估该节点的潜在结果。

4. 反向传播（Backpropagation）

- 将模拟结果反向传播到路径上的所有节点，更新这些节点的统计信息（如访问次数和胜率）。

归结原理

- 将待证明公式转换为**合取范式（CNF）**，即所有公式都表示为子句的合取形式。
- 将待证明公式的否定与已知条件一起构成一个子句集。
- 对子句集中任意两个子句应用**归结规则**，消解生成新的子句。
- 将新子句加入子句集中，重复这一过程，直到产生矛盾（空子句）或无新子句可生成。

- 若得到的归结式为==空集==则结论成立

机器学习

分类

分为监督学习、无监督学习、半监督学习、强化学习和自监督学习

监督学习

回归、分类、支持向量机、神经网络等

无监督学习

聚类

CNN

卷积神经网络 (CNN) 是为了 **图像处理** (尤其是图像识别和分类) 任务而发明的。

1. **任务目标**: CNN 最初是为了解决 **图像分类和物体识别** 问题。具体而言, 目标是通过从图像中提取特征, 实现对图像内容的自动分类。
2. **挑战**: 图像数据通常是高维的, 且具有空间结构 (如物体的位置、形状等), 传统的全连接神经网络难以处理这种结构化数据。传统方法需要手动提取特征, 而 CNN 通过卷积操作自动从图像中提取多层次的特征。
3. **需求**: 为了提高计算效率和准确性, 需要一种能够处理空间局部信息、共享权重、减少参数量且适应不同位置物体的模型。

结构:

1. **卷积层 (Convolutional Layer)**: 通过卷积操作提取输入图像的局部特征。该层使用多个卷积核 (滤波器) 对图像进行处理, 自动学习图像中的边缘、纹理、颜色等低级特征。
2. **池化层 (Pooling Layer)**: 池化操作 (如最大池化) 通过下采样减少图像的空间维度, 同时保留重要特征, 减少计算量。
3. **全连接层 (Fully Connected Layer)**: 在卷积和池化层提取到的高层特征基础上, 使用全连接层进行分类或回归等任务的输出。
4. **激活函数**: 如 ReLU 激活函数, 用于引入非线性, 使得模型能够学习更复杂的模式。

综上, CNN 的结构通过卷积、池化和全连接层的组合, 有效地解决了图像识别中的高维和局部特征提取问题。

RNN

循环神经网络（Recurrent Neural Network, RNN）主要是为了解决**序列数据**处理问题而发明的。传统的神经网络（如前馈神经网络、卷积神经网络）在处理时间序列、语音信号、文本等具有时序依赖的数据时，难以捕获输入之间的**时序关系和上下文依赖性**。RNN通过**循环连接**的结构，能够在隐藏层中保留历史信息，实现对序列数据的动态建模。

RNN在时间维度上共享参数，CNN在空间维度上共享参数。都是通过反向传播算法（RNN使用BPTT，CNN使用普通的反向传播）来最小化损失函数，从而学习任务的映射关系

RNN被广泛应用于以下任务：

- 1. 自然语言处理（NLP）
- 2. 语音识别
- 3. 时间序列预测
- 4. 视频分析
- 5. 生成模型

不同点

维度	循环神经网络（RNN）	卷积神经网络（CNN）
输入类型	主要处理序列数据（如文本、语音、时间序列）。	主要处理固定维度的结构化数据（如图像、视频）。
结构特点	含有循环连接，当前隐藏状态依赖于前一个时间步的状态。	卷积层和池化层用于提取局部特征，无时间依赖性。
主要任务	用于序列建模，捕获时间依赖和上下文关系。	用于图像、视频等静态数据的空间特征提取。
信息传播	信息在时间维度上传递，通过隐藏状态捕获历史信息。	信息在空间维度上传递，通过卷积核提取局部信息。
训练难点	训练时易出现梯度消失或梯度爆炸，尤其是处理长序列时。可使用改进的LSTM、GRU来缓解。	相对容易训练，梯度消失问题较少。
适用场景	NLP、语音识别、时间序列预测等序列任务。	图像分类、目标检测、视频处理等静态任务。
参数共享的方向	时间维度的参数共享（隐藏层参数在每个时间步相同）。	空间维度的参数共享（卷积核的参数在整个空间中共享）。

循环神经网络（RNN）的发明源于对序列数据（如语言、语音、时间序列）处理的需求。与卷积神经网络（CNN）相比，RNN擅长处理时间相关性，而CNN擅长捕捉空间相关性。两者各有优势，且在一些任务中可以结合使用（如视频分析中的时空特征建模）。

普通机器学习与深度学习

相同点：

1. **目标一致**：两者都通过学习数据中的模式来进行预测或分类。
2. **依赖数据**：数据的质量和数量都影响模型性能。
3. **优化算法**：两者都通过算法调整模型参数，最小化损失函数。

不同点：

1. **模型复杂性**：普通机器学习使用简单模型（如回归、决策树），深度学习使用复杂的神经网络（如CNN、RNN）。
2. **特征工程**：普通机器学习依赖手动特征选择，深度学习能够自动从数据中提取特征。
3. **数据需求**：深度学习需要大量数据，而普通机器学习适用于数据较少的情况。
4. **计算资源**：深度学习需要更强的计算资源（如GPU），普通机器学习要求较低。
5. **可解释性**：普通机器学习模型易于解释，深度学习则较为复杂且难以解释。
6. **应用场景**：普通机器学习适合结构化数据，深度学习擅长非结构化数据（如图像、文本）。

人工智能未来的发展方向

人工智能的未来发展将朝着更加智能化、普及化和融合化的方向发展。首先，**智能化**将更加深入各个领域，AI 将通过自我学习和推理能力提升，使得机器能够处理更加复杂的任务。例如，AI 在医疗、金融、教育等领域的应用将更加精细，提供个性化、实时的服务。其次，**普及化**将使 AI 技术深入到日常生活中，成为我们工作、学习和娱乐的自然组成部分。从智能家居到自动驾驶，AI 将进一步嵌入到我们的生活环境中，改变人们的生产和生活方式。最后，**融合化**将促进不同领域的AI技术跨界整合。例如，AI 和物联网（IoT）、大数据、5G 等技术的融合将推动智慧城市、智能制造等领域的创新发展，形成跨行业、跨领域的智能生态系统。